Lecture 6 · Part I · Lecture notes

Dimensionality reduction and unsupervised learning

Dataset: Olivetti faces, 400 photographs

How to use these notes

The primary text for this lecture is Géron, Chapters 7–8. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The first lecture with no target column **2**

2 Why anyone pays for it, and which tool a task calls for **3**

2.1 Four tasks from practice, and the one tool that fits each 3

3 The worked example: a photographic archive **4**

3.1 The brief 4

3.2 Before anything is fitted: can this work at all? 5

4 Derivation: PCA via the SVD **5**

4.1 Choosing d without guessing 6

5 Clustering, and how to choose k **7**

5.1 k-means, in five lines 7

5.2 Why inertia cannot choose k 7

5.3 Before reading any silhouette: what does nothing score? 8

5.4 The mean hides the shape 8

6 Putting the two halves together **9**

7 Beyond k-means **9**

7.1 DBSCAN 9

7.2 Anomaly detection with the tools already on the table 9

8 Spending the forty labels **9**

9 What to look out for in practice **10**

10 The notebook **11**

11 Design exercise: a grocery-delivery app's customer segments for its marketing team **11**

12 Exercises **13**

Summary **13**

---

1 The first lecture with no target column

Five lectures, and every one was handed the answers — most recently the cover types that Lecture 5's ensembles were scored against: a price for every district, a digit for every image, a survivor flag for every passenger, a cover type for every plot. Today there is nothing. That changes what a metric can be, what a baseline is, and how you find out you were wrong.

Supervised	**Unsupervised**
You are given $(\mathbf{x}^{(i)}, y^{(i)})$	you are given $\mathbf{x}^{(i)}$ only
The metric compares prediction to truth	the metric compares the grouping to itself
The baseline is "predict the majority"	the baseline is "group at random"
You know when you are wrong	you find out when someone looks

The consequence for the whole lecture

Every number computed today is a statement about *geometry*, not about people. Whether the geometry is the people is a separate question, and it is answered by looking.

2 Why anyone pays for it, and which tool a task calls for

Outside a benchmark, data is cheap and labels are not: every company has far more rows than it has verdicts about them. Three jobs recur, and every unsupervised method in this lecture does one of them.

The job	What it buys	The methods
Find the groups	a handful of segments to act on: one decision per group instead of one per row	k-means, DBSCAN
Fewer numbers per item	a model that trains faster, a plot you can look at, a store that fits	PCA, the SVD
Flag the unusual	the few rows unlike all the rest, before anyone has labelled a single one	DBSCAN's noise, a reconstruction error

A fourth, quieter use: structure found without labels tells you *which* few rows are worth paying to label. The worked example ends there.

2.1 Four tasks from practice, and the one tool that fits each

The point of the four is not the tool but the *mismatch*: on each task one method fits and the other three are visibly the wrong choice.

1. **A retailer's customer segments: k-means.** The marketing team wants a handful of segments, one campaign each, and every account placed in one; the rows are spend, order frequency, basket mix and time since the last order. k-means gives a few compact groups with every account assigned, and k is small because a person writes a campaign per segment: k is a business decision, which is exactly what k-means asks for. DBSCAN is the wrong tool because behaviour data has no empty space between groups — either most accounts come out as noise or everything is one cluster — and the team cannot send a campaign to "noise". PCA or the SVD give axes, not groups; a direction of spend is a description, not a segment anyone can write to. *Look out for:* the columns are in different units and k-means only sees distance, so scale first or the segments are "spend" and nothing else.

2. **A payments app's unusual transactions: DBSCAN.** The fraud team wants the few transactions unlike all the rest, every night, with no labels yet — the disputes that would label them arrive weeks later. Normal behaviour is dense and of any shape (commuters, weekend shoppers, subscriptions), and the output wanted is exactly DBSCAN's noise label: the points no dense region claims, with no k to choose. k-means is wrong because every transaction must belong to a cluster, so an anomaly becomes a far-off member of the nearest one and drags its centroid towards itself. PCA is wrong because a reconstruction error only catches what leaves the main subspace; a transaction that sits in a hole *inside* it scores as normal. *Look out for:* `eps` and `min_samples` together set the density a cluster must reach, and the fraction called noise is a decision the fraud team must own, not a number the algorithm found.

3. **Embeddings into a smaller model, and onto a plot: PCA.** A search team stores a long vector per product and per query; the downstream model trains too slowly, the index is too large, and nobody can look at the data. PCA is a linear map to fewer numbers that keeps

the directions where the vectors differ most, and states the price before you choose how many — the explained variance ratio. k-means is wrong because it replaces every vector by its centroid, so everything that distinguished two products in the same cluster is gone: quantisation, not compression. DBSCAN compresses nothing; it returns a label per row and keeps every column. *Look out for:* the two-dimensional plot is a projection and distances on it are not distances in the data; variance kept is not usefulness kept, so read what the kept directions are.

4. **Grouping unlabelled support tickets: the truncated SVD.** A support team has a backlog of tickets and no categories yet; each ticket is a sparse row of word counts, thousands of columns and almost all of them zero. The truncated SVD works on the sparse matrix as it is, and its leading directions are the recurring *themes* — the words that occur together; the tickets are then grouped in that small space. PCA is wrong because it centres first, which turns the sparse matrix into a dense one that does not fit in memory, for a mean that is mostly zeros. k-means on the raw counts groups by ticket length and the commonest words: in thousands of dimensions every ticket is about equally far from every other, and a distance that is the same everywhere ranks nothing. DBSCAN on the counts meets the same concentrated distance — no dense region, everything noise. *Look out for:* the themes are read by a person, and a direction that mixes two topics is not fixed by tuning, only by looking at its top words.

The tool follows the shape of the answer

Every item in one of a few compact groups: k-means, which assumes convex, roughly equal groups and a given k. Dense regions of any shape and a "none of these": DBSCAN, which assumes a density that separates groups from the rest. Fewer numbers per item with the price stated: PCA, which assumes the useful variation is where the variance is. Themes in a sparse table of counts: the truncated SVD, the same assumption without centring. Choose by what the consumer of the output can do with it — a segment nobody can write to, a noise label nobody can act on and a component nobody can read are all technically correct and all useless.

The rest of these notes works one example in full, because every one of these tools has the same failure mode: it does exactly what it was asked, in a geometry that may not be the one you meant.

3 The worked example: a photographic archive

3.1 The brief

"We have a photographic archive. We do not know how many distinct people are in it. Group the photographs by person, so that a human can name a group instead of naming every photograph." Nobody remembers who is in it, an annotator costs money per photograph, and we are funded for *forty* labels. That is the ordinary situation outside a benchmark: data is cheap, labels are not.

400 photographs of 40 people, ten each, 64×64 pixels flattened to $\mathbf{x}^{(i)} \in \mathbb{R}^{4096}$ with $x_j \in [0, 1]$ — 6.6 MB for the whole corpus. Ten times more dimensions than points.

The rule that keeps this honest

Olivetti ships with the identity of every photograph; the archive we are pretending to hold does not. So y is used in exactly two ways, both announced where they happen: as the *forty labels we paid for*, and as an *audit*, to find out what those forty could not have told us. Any other use of y is the same error as scoring on the test set, wearing different clothes.

3.2 Before anything is fitted: can this work at all?

k-means groups points close in Euclidean distance, so one measurement decides whether the rest of the lecture can work. It is an audit — it uses the hidden labels.

Euclidean distance, raw pixels	Pairs	Mean	Median
Same person	1,800	8.38	8.15
Different people	78,000	12.41	12.15

The gap is real and it is small: 6% of same-person pairs sit further apart than the *median* different-person pair.

Failure condition — stated in advance

No method that groups by Euclidean distance in raw pixels will recover the forty people cleanly. Not because the method is bad — because the representation puts a lamp and a nose on the same axis, with the same weight.
Moving the light source changes hundreds of pixel values by a large amount. Changing the person changes a few dozen by a small one. Euclidean distance adds them all up equally. Measuring this costs one cell and tells you whether you are about to tune or about to hope.

4 Derivation: PCA via the SVD

The question: among all d-dimensional subspaces, which loses the least when you project onto it, and how much does it lose?

Write $\mathbf{X} \in \mathbb{R}^{m \times n}$ with one photograph per row, $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^\top$ the centred data, $\mathbf{W} \in \mathbb{R}^{n \times d}$ with $\mathbf{W}^\top\mathbf{W} = \mathbf{I}_d$, and $\mathbf{P} = \mathbf{W}\mathbf{W}^\top$ the orthogonal projector, and $\|\mathbf{A}\|_F^2 = \sum_{ij} a_{ij}^2 = \operatorname{tr}(\mathbf{A}^\top\mathbf{A})$. There is no probability in this derivation and none is needed.

Failure condition — an SVD taken without centring

Without centring, the direction of largest $\sum_i \|\mathbf{w}^\top\mathbf{x}^{(i)}\|^2$ is essentially $\bar{\mathbf{x}}$ itself, and the first component describes nothing but "this is a photograph of a face".
Scikit-Learn's `PCA` centres for you. If you write the SVD by hand it does not, and the result is a plausible-looking first component and no error message. It is the commonest silent bug in a hand-rolled PCA.

Seven steps

1. The objective. Project each centred face and measure how far it had to move:

$$E(\mathbf{W}) = \sum_i \left\|\tilde{\mathbf{x}}^{(i)} - \mathbf{W}\mathbf{W}^\top\tilde{\mathbf{x}}^{(i)}\right\|_2^2 = \left\|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{W}\mathbf{W}^\top\right\|_F^2.$$

2. Pythagoras. $\mathbf{P}$ is an orthogonal projector, so kept and lost are orthogonal, and since $\|\tilde{\mathbf{X}}\|_F^2$ does not depend on $\mathbf{W}$,

$$E(\mathbf{W}) = \|\tilde{\mathbf{X}}\|_F^2 - \|\tilde{\mathbf{X}}\mathbf{W}\|_F^2.$$

3. As a trace. $\|\tilde{\mathbf{X}}\mathbf{W}\|_F^2 = \operatorname{tr}(\mathbf{W}^\top\mathbf{C}\mathbf{W})$ with $\mathbf{C} = \tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$, which is $n \times n$, symmetric, positive semi-definite, and $m - 1$ times the sample covariance matrix.

4. The SVD diagonalises it. With $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$,

$$\mathbf{C} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top,$$

so the columns of $\mathbf{V}$ are the eigenvectors of $\mathbf{C}$ with eigenvalues σ_j^2. They are the principal components.

5. Rotate the unknown. Write $\mathbf{W} = \mathbf{V}\mathbf{A}$ with $\mathbf{A}^\top\mathbf{A} = \mathbf{I}_d$; then $\operatorname{tr}(\mathbf{W}^\top\mathbf{C}\mathbf{W}) = \sum_j \sigma_j^2 h_j$, where h_j is the squared norm of row j of $\mathbf{A}$. Both constraints are one line each: $\sum_j h_j = \|\mathbf{A}\|_F^2 = \operatorname{tr}(\mathbf{A}^\top\mathbf{A}) = d$, and $h_j \le 1$ because h_j is the squared length of the projection of a unit basis vector onto d orthonormal columns.

6. A weighted sum with d units to spend. The σ_j^2 are sorted, so put a full unit on each of the d largest and nothing on the rest: the maximum is $\sum_{j \le d} \sigma_j^2$, attained at $\mathbf{W} = \mathbf{V}_d$, the first d columns of $\mathbf{V}$. Any other allocation moves weight from a larger σ_j^2 to a smaller one.

7. The error that is left. Since $\|\tilde{\mathbf{X}}\|_F^2 = \operatorname{tr}(\mathbf{C}) = \sum_j \sigma_j^2$,

$$\min_{\mathbf{W}} E(\mathbf{W}) = \sum_{j>d} \sigma_j^2.$$

That is Eckart–Young–Mirsky in the Frobenius norm. "PCA minimises reconstruction error" is a *theorem*, not a slogan — and it names the price in advance: the tail of the spectrum, readable before you choose d.

Note also that step 2 says minimising error and maximising projected variance are *the same problem*, not two related ones.

4.1 Choosing d without guessing

By step 7 the fraction of squared error removed by keeping d components is exactly $\sum_{j \le d} r_j$, where $r_j = \sigma_j^2 / \sum_i \sigma_i^2$. The explained variance ratio is not a heuristic; it is the theorem, normalised.

Cumulating to a target, **123 components reach 95%**. The rest of this lecture runs at 123 — a factor of 33 fewer numbers per photograph. In code that is `PCA(n_components=0.95)`: a float between 0 and 1 is read as a variance target and scikit-learn chooses d from the cumulative ratio, while an integer means "this many components" — so `n_components=1` and `n_components=1.0` do completely different things.

A principal component is a vector in $\mathbb{R}^{4096}$, so reshaping it to 64×64 makes it an image; on faces these are called **eigenfaces**, and every photograph in the corpus is the mean face plus a weighted sum of them. That is not a property of faces but of PCA: the components are directions in the input space, so whatever an input looks like, a component looks like too. And looking at the first few, they are *lighting* — exactly what the distance histogram predicted would dominate. A held-out face rebuilt from its components is recognisable well before 123 of them — identity survives a compression the pixel values do not.

5 Clustering, and how to choose k

5.1 k-means, in five lines

1. Choose k initial centroids; scikit-learn uses **k-means++**, which spreads them out.
2. Assign every point to its nearest centroid.
3. Move every centroid to the mean of its assigned points.
4. Repeat 2–3 until no assignment changes.
5. Repeat the whole thing `n_init` times and keep the best.

Step 5 is not optional. Lloyd's algorithm converges — to a local minimum that depends entirely on where you started. `n_init` changed its default between scikit-learn versions, so name it.

k-means makes four assumptions whether you asked or not: the partition is a Voronoi diagram, so every cluster is *convex*; the only quantity is distance to a centre, so clusters are *spherical and equal-sized*; every point belongs to exactly one cluster, so there is no "none of these"; and k *is given*. Three of the four are false here. The first three are dropped by DBSCAN, later in the lecture, and are why DBSCAN was the tool for the transactions; the fourth, choosing k, is what the rest of this section is about.

5.2 Why inertia cannot choose k

$$J(k) = \sum_i \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{c(i)} \right\|_2^2$$

is what k-means minimises — and it is non-increasing in k, with $J(m) = 0$ exactly. Refining a partition cannot increase the sum of squared distances to the assigned centre. *There is no interior minimum to find, at any k, on any dataset*, so "minimise inertia" selects one cluster per photograph. The elbow rule exists to work around that, and on our curve the kneedle rule — the point furthest below the chord, both axes rescaled — picks $k = 19$ where the archive has 40 — with 46% of the inertia at $k = 2$ still present at $k = 40$.

The elbow is not a criterion

It is a picture of a criterion that does not exist, and no better heuristic repairs it, because the information is not in J.

The silhouette does have an interior maximum. For a point i, with a its mean distance to its own cluster and b to the nearest other,

$$s_i = \frac{b - a}{\max(a, b)} \in [-1, 1] :$$

near +1 the point is far inside its own cluster, near 0 it is on a boundary, near −1 it is assigned to the wrong cluster. The silhouette score is the mean of s_i, penalised both for splitting a group and for merging two. It costs $O(m^2 n)$ — every pair, over every feature, and both factors matter: the archive is bigger than 400, and n is larger than it needs to be.

5.3 Before reading any silhouette: what does nothing score?

Random assignment	**Silhouette**
$k = 10$	-0.0422 ± 0.0045
$k = 40$	-0.1288 ± 0.0107
$k = 60$	-0.1805 ± 0.0124

The anchor sits at or below zero, and now you know the width of the noise around it. A silhouette of 0.02 is not a discovery.

The silhouette curve has a maximum, well above the anchor. The maximum is also low, and the curve near it nearly flat, so the chosen k is not decisively better than its neighbours.

5.4 The mean hides the shape

A mean silhouette means nothing until you look. The audit below counts what looking shows, with one more definition: the **purity** of a cluster C is the share of its members that belong to its most common identity, $\max_y |\{i \in C : y_i = y\}| / |C|$ — 1.00 when every photograph is the same person — and mean purity averages it over the clusters, each counting once. It needs every identity, so it is an audit, and it is read beside the cluster sizes: a cluster of one photograph is pure by construction, so purity alone rewards over-splitting exactly as inertia does.

At $k = 40$, against the hidden identities	**Value**
Cleanest cluster: 10 photographs, purity	1.00
Worst cluster: 24 photographs of 10 people, purity	0.21
Mean purity across the forty clusters	0.74
Clusters holding exactly one person	16
Clusters with exactly ten members	7
Cluster sizes, smallest to largest	4 to 25

Every true group has exactly ten members. The overlap the distance histogram showed before anything was fitted is now a count — and the worst cluster is not a person at all, but a lighting condition: several different people lit and posed the same way.

6 Putting the two halves together

The clustering code is unchanged — same grid, same seed, same `n_init`. Only what k-means is looking at changed.

Both cost models lose a factor of 33 in their n: the sweep runs at 123 dimensions instead of 4,096.

7 Beyond k-means

k-means assumes convex, equal-sized clusters, total assignment and a known k. DBSCAN drops the first three.

7.1 DBSCAN

DBSCAN rests on three definitions. A point is a **core point** if at least `min_samples` points lie within `eps` of it. Core points within `eps` of each other are in the same cluster, and so are the non-core points in their neighbourhoods. Everything else is **noise**, labelled -1 — so count clusters as `len(set(db.labels_) - {-1})`. DBSCAN has no `predict`: it is *transductive*, and to label a new point you fit again or train a classifier on its output.

7.2 Anomaly detection with the tools already on the table

By reconstruction error: an image the eigenfaces cannot rebuild, `pca.inverse_transform(pca.transform(X))`, is unlike the images that built them, so fit the PCA on the clean corpus, score every image and flag the worst few. By density: DBSCAN's noise points are the images no dense region claims — the transactions example again, on faces. The notebook tests the first by planting twelve corrupted images — four rotated, four dimmed, four double-exposed — and counting how many land in the top twelve, with the detector fitted on the clean corpus only, so it has not learned the images it is asked to find.

Two things to look out for. A detector fitted on data that already contains the anomalies has learned them. And the threshold — how many to flag — is a decision about the cost of a missed one against the cost of a false alarm, not a number the method knows.

8 Spending the forty labels

Split the corpus 280 / 120, stratified by identity, and label forty of the 280 training photographs. Where they are spent turns out to matter more than what is done with them. Forty at random; forty chosen as the photograph nearest each of forty cluster centroids (`km.transform(Z).argmin(axis=0)`); the representative's label propagated to its whole cluster, 280 labels from 40; propagated only to the 75% of each cluster closest to its centroid; and, as the ceiling the project cannot buy, all 280 true labels. Same classifier, same held-out faces, twenty splits; the only thing that varies is which forty were labelled.

Labels used	Held-out accuracy	sd over 20 splits
40 at random	48.1%	3.8%
40, one per cluster	60.5%	4.8%
Propagated to the whole cluster	55.6%	4.8%
Propagated to the closest 75%	55.1%	4.9%
All 280 true labels — the ceiling	97.4%	1.6%

Twenty splits, because the differences between the middle three rows are of the same order as their spread, and a comparison that turns on less than the spread is not a comparison. *Propagation buys 280 labels from 40 and does worse than the 40 alone*, because a wrong propagated label is worse than no label. Report that; do not bury it — it is the commonest shortcut in a labelling budget, and it is a loss.

9 What to look out for in practice

The slides close with two tables; here they are as prose, in the order the work happens.

- **Columns in different units.** Distance is owned by the largest unit and the segments are "spend". Scale before any distance, and keep the scaler with the model.
- **Something dominates the variation.** The clusters and the leading components are that thing — a lamp, a season, a device. Measure first whether distance tracks what you care about; change the representation, not the algorithm.
- **`n_components` as an integer or a float.** A count or a variance target, one character apart. Read it aloud in every pipeline you inherit.
- **The two-dimensional plot.** A projection: distances on it are not distances in the data. Use it to look, never to measure.
- **Nobody told you k.** Inertia always prefers more and the silhouette is flat near its maximum. Read it against a random anchor, then let the consumer choose among the flat candidates.
- **A clean mean silhouette.** One cluster can be a lighting condition, a region, a device. Look at a sample of every cluster, worst first, before naming any.
- **The job reruns next month.** Different seed, different segments, different names. Fix `n_init` and `random_state`, and check stability across resamples before trusting a segment.
- **DBSCAN after rescaling.** `eps` was set in the old units. Treat it as part of the representation, and remember DBSCAN cannot `predict`.
- **Anomalies in the training data.** The detector has learned them. Fit on data believed clean; the threshold is a cost decision.
- **A labelling budget.** Propagated labels cost more than they buy. Spend labels on representatives, and report the loss rather than bury it.

10 The notebook

What to change

Change `n_components=0.95` to `0.99`, which takes d from 123 to 260. The sweep slows by roughly the ratio of the two and what k-means finds barely moves — the whole trade-off in one edit.

11 Design exercise: a grocery-delivery app's customer segments for its marketing team

The lecture closes with a design exercise in the manner of an ML system-design interview, on the system behind a company's product, with the shape of data this lecture's methods suit.

The task. A grocery-delivery app holds every account's order history. Its marketing team wants a handful of customer segments — one campaign per segment, written by a person and sent through the app — and every account's segment. The business reads retention and promotion spend per order; a campaign aimed at a segment that is not one wastes both, and no account comes with a label. Given: for each account, its orders over the last year — what, when, how much, on promotion or not — and nothing else. Millions of accounts, dozens of departments, and a budget to interview a few hundred customers. The output feeds the marketing team, who name and write to each segment, the campaign tool that assigns a new account, and finance, who track the spend.

1. **Frame it honestly.** One row per account; the output is a segment for every account and a profile of each segment in the measured columns. There is no truth to compare with, so the metric compares the grouping to itself — the silhouette, read against what a random grouping scores — and to its own stability under resampling; the baseline is "group at random", and the test is whether a marketer reading a segment's baskets sees one kind of customer. *Pitfall:* the first attempt asks the model how many kinds of customer there are, reads the answer as a fact about people, and writes a campaign per cluster. Every number here is a statement about geometry; decide before fitting what will be looked at — the baskets nearest each centre, and the worst segment as well as the best.

2. **The row, scaled before any distance.** One account is a vector of shares and rates — the share of spend per department, orders per week, basket size, the share on promotion — so that how much a household spends is not an axis of its own; and k-means adds every column up with the same weight, so standardise with a scaler fitted on the training accounts before the first distance. *Pitfall:* annual spend in euros beside orders per week, and the column with the largest unit decides every distance: the "segments" are bands of total spend, a campaign finance could have written from one column. Remove the nuisance at the row, not by deleting leading components afterwards: that threw away signal with the nuisance when it was measured today.
   item
   textbfPCA, and how many components. Dozens of correlated department columns: centre, take the SVD, and let a float to `n_components` choose d from the cumulative variance ratio. A

component is a direction in the columns, so it can be read — fresh against packaged, evening top-ups against one weekly shop — and both cost models are linear in n, so the sweep over k and the silhouette get cheaper by the same factor. *Pitfall:* variance kept is not usefulness kept. The first component of a table of departments is usually "how much" — total spend wearing forty names — which the shares and rates of the previous step were meant to remove. Read every kept component in the columns before clustering on it; if the first one is size, the scaling is wrong, and no choice of d repairs it.

3. **k-means, and what it assumes.** Lloyd's algorithm from k-means++ starts, with `n_init` named and the best of several restarts kept. It assumes convex, spherical, equal-sized clusters, total assignment and a given k — and the app's customers are one large crowd of weekly shoppers with a few small, distinct groups around it. *Pitfall:* Voronoi cells slice the crowd into arbitrary wedges and absorb the small groups, the ones a campaign could actually speak to, and every account gets a segment. Read the cluster sizes and the silhouette diagram first; if the shape is wrong, a Gaussian mixture fits ellipsoids of different sizes, and DBSCAN finds dense groups with a noise label, at the price of no `predict` for a new account.

4. **Choose k.** Inertia is non-increasing in k and zero at one cluster per account, so it cannot choose, and the elbow is a picture of a criterion that does not exist. The silhouette has an interior maximum; quadratic in the rows, it is computed on a sample of the accounts in the reduced space, and before reading any value the accounts are assigned to k groups at random, many times, for the null and its width. *Pitfall:* a silhouette with no anchor has no units, and the marketing team is told that "the data shows five segments" — and budgets five campaigns — on a maximum that is low and nearly flat across its neighbours. Report the curve, the anchor and the width of the noise, and say so when the chosen k is not decisively better than its neighbours.

5. **Stability across resamples.** Refit at the chosen k on several random subsamples, match the segments of two fits by overlap, and count the share of the accounts they share that keep their segment. *Pitfall:* the campaign is written and the next refit — a new quarter of orders, a different seed — produces segments that do not match, so the message written for one segment reaches accounts no longer in it. Restarts remove the variation that is Lloyd's algorithm; what remains is the data's, and a k whose partitions disagree is unsupported whatever the silhouette says.

6. **Name a segment without inventing a story.** For each segment, its profile in the original units against the mean of all accounts, and the baskets of the accounts nearest its centroid; spend the interview budget on those representatives rather than on accounts drawn at random, and show the team the worst segment as well as the cleanest. *Pitfall:* "young professionals" is written on a centroid nobody looked at and a campaign goes out in that voice, when the segment is "accounts delivering to an office at lunchtime" — a condition, this problem's lighting, not a kind of person. Name a segment only in the columns that were measured, and never propagate an interviewed answer to its whole segment: a wrong label is worse than none.

7. **In production.** k-means assigns every account, so score each one first — the density under a Gaussian mixture, or the PCA reconstruction error — and give the lowest few no segment: a shared flat's account, a café buying for resale. Release the segments to the campaign tool with a holdout of accounts that receive no campaign, so that each campaign's lift on retention is

measured against it; refit on a rolling window as the season turns, compare the new partition with the old by the share of accounts that keep their segment, and recompute the anchor, with the data team on call for the refit. *Pitfall:* segments fitted in summer and a campaign that runs in the winter holidays; the scaler, the components and the centres describe a season that has gone, and the share of accounts that change segment on refit is the only sign. Monitor it, and tell the team which segments survived.

8. **Ship it.** Order log → one row per account, shares and rates → held-out accounts → scaler and PCA in a pipeline → k-means with restarts → silhouette against the random anchor → stability across resamples → profiles and representatives, named in the measured columns → anomaly scores → released with a holdout; refit by season and compare.

12 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 7's deck and in the exercise book.

1. A marketing team wants every customer placed in one of a handful of segments; a fraud team wants the few transactions unlike all the rest, with no labels yet. Name the clustering method for each, and say why the other one is the wrong tool in each case. [5]

2. The explained variance ratio is sometimes called a heuristic for choosing d. Say why it is not one, and what keeping 95% of the variance leaves as reconstruction error. [5]

3. A customer-segmentation job reruns every month on fresh orders, and the marketing team complains that the segments "moved": the groups and their sizes differ from last month's. Name two things to fix in the code, and one check to run before any segment is trusted. [4]

4. Inertia always falls as k rises, so it cannot choose k. State what silhouette adds, and one situation where it also fails. [5]

5. You have 4,096-dimensional face vectors and are told two faces are "close". Explain why that alone does not mean they are the same person. [4]

Summary

Unsupervised learning finds groups, cuts the numbers per item and flags the unusual, with no label to score against: every number is about geometry, and you find out you were wrong by looking. The tool follows the shape of the answer — every item in a compact group, k-means; dense regions of any shape and a "none of these", DBSCAN; fewer numbers with the price stated, PCA; themes in sparse counts, the truncated SVD — and each is the wrong tool for the other three jobs.

PCA is the SVD of the *centred* data: the principal subspace minimises squared reconstruction error exactly, the error it leaves is $\sum_{j>d} \sigma_j^2$, and the explained variance ratio reads it before you choose d — 123 components hold 95% of 4,096 pixels. Inertia cannot choose k; the silhouette can, read against a random anchor and then against the items themselves, because a mean hides a

cluster that is a lighting condition rather than a person: mean purity 0.74 at $k = 40$, and the worst cluster not a person at all.

Scale before any distance; fix the seed; look at the worst cluster first; fit a detector on clean data; the threshold and k are decisions the consumer owns. Forty labels spent on cluster representatives were worth a quarter more accuracy than forty spent at random, and propagating them to 280 was worse than not propagating at all.

The closing design exercise built a grocery-delivery app's customer segments for its marketing team in nine steps — shares and rates scaled before any distance, PCA read in the columns, k-means read against its assumptions, k by silhouette against a random anchor, stability across resamples, names in the measured columns, anomalies by density, a holdout and a refit by season — each step named for the pitfall it avoids.